



Universidad Peruana de Ciencias Aplicadas

FACULTAD DE INGENIERÍA

CURSO

Lenguaje de programación

Profesor

Cristian Roberto Sánchez Flores

NRC: 12795

Integrantes

Escobar Cory, Miguel Andrée (U20251O828)

Ramírez Huamani, Mao Lenon (U20241J117)

Rodriguez Quispe, Dario Francisco (U202523240)

Zelada Gutarate, Dayara Priyanka (U20251L120)

2026

Índice

1. Objetivo del Estudiante (STUDENT OUTCOME)	3
1.1. Student OutCome ABET 1 (EAC)	3
1.2. Student OutCome ABET 1 (CAC)	4
2. Capítulo 1: Lista de Requerimientos	7
2.1. Nombre del proceso	7
2.2. Descripción del proceso	7
2.3. Objetivos Generales	8
2.4. Objetivos Específicos	8
2.5. Problema Actual	8
2.6. Solución Propuesta	9
2.7. Beneficios del Sistema	9
2.8. Alcance del Sistema	11
2.9. Requerimientos Funcionales	12
2.10. Ejemplo Reales	14
2.11. Algoritmo General del Proceso Principal	15
3. Capítulo 2: Estructuras, Archivos y Funciones	15
3.1. Estructura de Datos	15
3.2. Archivos de Persistencia de Datos	17
3.3. Funciones Principales del Sistema	18
3.4. Relación entre requerimientos y funciones	20
3.5. Librerías utilizadas	21
4. Capítulo 3: Listado de funcionalidades cumplidas y Manual de Usuario	21
4.1. Listado de funcionalidades cumplidas	21
4.2. Manual de Usuario	23
4.2.1. Presentación del proyecto	23
4.2.2. Menús	23
4.2.3. Menú Administrador - Funciones	24
4.2.4. Menú Estudiante - Funciones	32
4.2.5. Validaciones y restricciones	34
4.2.6. Salir	35
5. Conclusiones	36
6. Recomendaciones	37
7. Bibliografía y Citas	38
8. Participación	38

1. Objetivo del Estudiante (STUDENT OUTCOME)

1.1. Student OutCome ABET 1 (EAC)

ABET 1 La capacidad de identificar, formular y resolver problemas complejos de ingeniería aplicando los principios de ingeniería, ciencia y matemática.	
Escobar Cory, Miguel Andrée	
c1. Identifica problemas de ingeniería de sistemas de información.	Identifiqué la necesidad de centralizar el control de rutas, horarios y capacidad de las unidades de transporte universitario en un solo sistema.
c2. Formula problemas de ingeniería de sistemas de información aplicando ciencias, matemáticas e ingeniería.	Formulé la solución aplicando estructuras, arreglos dinámicos y archivos de texto para organizar los datos de rutas y unidades.
c3. Resuelve problemas de ingeniería de sistemas de información aplicando ciencias, matemáticas e ingeniería.	Resolví el problema dividiendo el sistema en funciones independientes para el registro de rutas, validando los datos ingresados por el usuario.
Ramírez Huamani, Mao Lenon	
c1. Identifica problemas de ingeniería de sistemas de información.	Identifiqué los elementos principales del problema, como las rutas, los horarios, las unidades y los pasajeros que generan la sobrecarga del servicio.
c2. Formula problemas de ingeniería de sistemas de información aplicando ciencias, matemáticas e ingeniería.	Formulé la solución utilizando una matriz dinámica bidimensional para representar la ocupación de pasajeros por ruta y horario.
c3. Resuelve problemas de ingeniería de sistemas de información aplicando ciencias, matemáticas e ingeniería.	Resolví el problema mediante funciones que actualizan la matriz de horarios y muestran la ocupación de cada ruta de forma ordenada.
Rodriguez Quispe, Dario Francisco	
c1. Identifica problemas de ingeniería de sistemas de información.	Identifiqué la necesidad de automatizar el registro de pasajeros y el control de la capacidad máxima de cada ruta.

c2. Formula problemas de ingeniería de sistemas de información aplicando ciencias, matemáticas e ingeniería.	Formulé la solución aplicando estructuras para conductores y unidades, y funciones para validar la disponibilidad de recursos.
c3. Resuelve problemas de ingeniería de sistemas de información aplicando ciencias, matemáticas e ingeniería.	Resolví el problema mediante funciones que validan la disponibilidad antes de asignar unidades y conductores a una ruta.
Zelada Gutarate, Dayara Priyanka	
c1. Identifica problemas de ingeniería de sistemas de información.	Identifiqué la necesidad de generar reportes y estadísticas confiables sobre la demanda del servicio de transporte.
c2. Formula problemas de ingeniería de sistemas de información aplicando ciencias, matemáticas e ingeniería.	Formulé la solución aplicando arreglos dinámicos y funciones de cálculo para generar el ranking y el porcentaje de ocupación.
c3. Resuelve problemas de ingeniería de sistemas de información aplicando ciencias, matemáticas e ingeniería.	Resolví el problema mediante funciones que calculan y muestran las estadísticas y el ranking de rutas a partir de los datos almacenados.

1.2. Student OutCome ABET 1 (CAC)

ABET 1 La capacidad de analizar un problema complejo y aplicar principios de computación y otras disciplinas relevantes para identificar soluciones.	
Escobar Cory, Miguel Andrée	
c1. Identifica los componentes externos e internos presentes en un problema complejo en sistemas de información.	Reconocí los componentes internos del sistema, como las rutas, los horarios, las unidades, los conductores y los archivos de texto que conservan la información.
c2. Identifica las dependencias y/o conexiones que existen entre cada uno de los componentes	Analicé las dependencias entre los módulos, ya que registrar una ruta depende de validar su

del problema complejo aplicando los principios de computación, ciencia y matemática.	código y asignar una unidad depende de su disponibilidad.
c3. Emite conclusiones sobre el impacto de cada componente interno y/o externos en el problema complejo de sistemas de información y emite soluciones aplicando principios de computación, ciencia y matemática.	Concluí que dividir la lógica en funciones independientes facilita el mantenimiento del sistema y reduce errores al registrar rutas y unidades.
Ramírez Huamani, Mao Lenon	
c1. Identifica los componentes externos e internos presentes en un problema complejo en sistemas de información.	Reconocí los componentes principales del sistema, como las rutas, la matriz de horarios, las unidades y los pasajeros registrados.
c2. Identifica las dependencias y/o conexiones que existen entre cada uno de los componentes del problema complejo aplicando los principios de computación, ciencia y matemática.	Analicé cómo la matriz de horarios se conecta con las rutas registradas, ya que la ocupación depende directamente de la capacidad de cada ruta.
c3. Emite conclusiones sobre el impacto de cada componente interno y/o externos en el problema complejo de sistemas de información y emite soluciones aplicando principios de computación, ciencia y matemática.	Concluí que representar la ocupación mediante una matriz dinámica mejora el análisis de la demanda y facilita mostrar la información ordenada.
Rodriguez Quispe, Dario Francisco	
c1. Identifica los componentes externos e internos presentes en un problema complejo en sistemas de información.	Reconocí los componentes del sistema relacionados con el control de capacidad, como las unidades, los conductores y las rutas asignadas.
c2. Identifica las dependencias y/o conexiones que existen entre cada uno de los componentes del problema complejo aplicando los principios de computación, ciencia y matemática.	Analicé las conexiones entre unidades, conductores y rutas, ya que una asignación depende de la disponibilidad de los recursos registrados.

c3. Emite conclusiones sobre el impacto de cada componente interno y/o externos en el problema complejo de sistemas de información y emite soluciones aplicando principios de computación, ciencia y matemática.	Concluí que validar la disponibilidad antes de asignar recursos evita inconsistencias entre unidades, conductores y rutas.
Zelada Gutarate, Dayara Priyanka	
c1. Identifica los componentes externos e internos presentes en un problema complejo en sistemas de información.	Reconocí los componentes del sistema orientados a la generación de reportes, como las estadísticas, el ranking y los archivos de almacenamiento.
c2. Identifica las dependencias y/o conexiones que existen entre cada uno de los componentes del problema complejo aplicando los principios de computación, ciencia y matemática.	Analicé cómo las estadísticas y el ranking dependen de los datos almacenados en rutas, horarios y pasajeros para generar resultados confiables.
c3. Emite conclusiones sobre el impacto de cada componente interno y/o externos en el problema complejo de sistemas de información y emite soluciones aplicando principios de computación, ciencia y matemática.	Concluí que conservar la información en archivos de texto permite generar reportes confiables y facilita el cumplimiento de los requerimientos planteados.

2. Capítulo 1: Lista de Requerimientos

2.1. Nombre del proceso

Sistema inteligente de control de transporte universitario.

2.2. Descripción del proceso

El Sistema Inteligente de Control de Transporte Universitario es una solución informática diseñada para optimizar la administración del servicio de transporte de la Universidad Peruana de Ciencias Aplicadas (UPC). El sistema permitirá gestionar de manera centralizada la información relacionada con las rutas, paraderos, horarios, pasajeros y capacidad de las unidades de transporte utilizadas para el traslado de estudiantes entre las diferentes sedes universitarias.

Actualmente, la gestión del transporte universitario puede presentar dificultades relacionadas con la alta demanda de pasajeros, la distribución inadecuada de rutas, la limitada capacidad de algunas unidades y la falta de información organizada para la toma de decisiones. Estas situaciones pueden generar retrasos, sobrecarga de pasajeros y una disminución en la calidad del servicio brindado a la comunidad estudiantil.

Frente a esta problemática, el sistema permitirá automatizar procesos fundamentales como el registro de rutas, la administración de horarios, el control de capacidad de los buses y el registro de pasajeros. Asimismo, facilitará la generación de reportes estadísticos que permitan identificar las rutas con mayor demanda, los horarios más concurridos y el nivel de ocupación de cada unidad de transporte.

La información gestionada por el sistema será almacenada en archivos de texto, permitiendo conservar los registros para futuras consultas y análisis. De esta manera, se contribuirá a mejorar la organización, eficiencia y control del servicio de transporte universitario, brindando información confiable para la toma de decisiones administrativas.

Según PIARC (2023), los Sistemas de Transporte Inteligente permiten a los usuarios estar mejor informados y garantizan un uso más seguro, coordinado e inteligente de las redes de transporte.

2.3. Objetivos Generales

Desarrollar un Sistema Inteligente de Control de Transporte Universitario que permita administrar de manera eficiente las rutas, horarios, pasajeros y capacidad de las unidades de transporte de la Universidad Peruana de Ciencias Aplicadas (UPC), mediante la automatización de procesos de registro, control y generación de reportes, contribuyendo a mejorar la organización, supervisión y gestión del servicio de transporte universitario.

2.4. Objetivos Específicos

- Registrar y administrar la información de las rutas universitarias, incluyendo sus paraderos, capacidad máxima y estado operativo.
- Gestionar los horarios de salida y llegada de las diferentes rutas mediante estructuras organizadas que faciliten su consulta y actualización.
- Registrar a los pasajeros que utilizan el servicio de transporte universitario y asociarlos a las rutas correspondientes.
- Controlar la capacidad de cada ruta para evitar sobrecargas y garantizar una adecuada distribución de pasajeros.
- Generar estadísticas y reportes relacionados con la demanda de rutas, horarios y capacidad utilizada.
- Almacenar y recuperar información utilizando archivos de texto para garantizar la persistencia de los datos registrados.
- Contribuir a la mejora de la organización y eficiencia del servicio de transporte universitario mediante la automatización de procesos administrativos.

2.5. Problema Actual

Actualmente, el servicio de transporte de la Universidad Peruana de Ciencias Aplicadas (UPC) presenta diversas dificultades relacionadas con la gestión y organización de las rutas universitarias. El incremento de estudiantes que utilizan este servicio genera una mayor demanda en determinados horarios, ocasionando saturación de pasajeros en algunas unidades de transporte y una distribución poco eficiente de los recursos disponibles.

Entre los principales problemas identificados se encuentran la insuficiente cantidad de rutas entre sedes, la limitada disponibilidad de paraderos, la dificultad para controlar la capacidad máxima de los buses y la falta de mecanismos automatizados para registrar y supervisar la información relacionada con pasajeros, horarios y rutas.

Además, gran parte de la administración de estos procesos se realiza de manera manual, lo que incrementa la posibilidad de errores, retrasa la obtención de información y dificulta la generación de reportes para la toma de decisiones. Como consecuencia, pueden producirse demoras en los traslados,

desorganización en la asignación de pasajeros y una disminución en la calidad del servicio ofrecido a los estudiantes.

Por ello, surge la necesidad de implementar una herramienta informática que permita centralizar, controlar y optimizar la gestión del transporte universitario mediante procesos automatizados y reportes oportunos.

2.6. Solución Propuesta

Se propone desarrollar un Sistema Inteligente de Control de Transporte Universitario que permita gestionar de manera eficiente la información relacionada con rutas, horarios, paraderos, pasajeros y capacidad de las unidades de transporte.

El sistema permitirá registrar y administrar las rutas universitarias, controlar la capacidad máxima de cada unidad, gestionar los horarios de operación y registrar a los pasajeros que utilizan el servicio. Asimismo, incorporará mecanismos de validación para evitar sobrecargas de pasajeros y garantizar una mejor distribución de los usuarios entre las diferentes rutas disponibles.

Adicionalmente, la solución generará reportes estadísticos que facilitarán el análisis de la demanda del servicio, permitiendo identificar las rutas más utilizadas, los horarios con mayor concurrencia y la capacidad disponible en cada unidad de transporte. Toda la información será almacenada en archivos de texto para garantizar su disponibilidad y consulta posterior.

La implementación de esta herramienta contribuirá a mejorar la organización del transporte universitario, optimizar el uso de los recursos disponibles y brindar información confiable para apoyar la toma de decisiones administrativas.

2.7. Beneficios del Sistema

- **Automatización de los procesos administrativos**
El sistema automatiza el registro y la gestión de rutas, horarios, pasajeros y recursos del transporte universitario, reduciendo el trabajo manual, minimizando errores y mejorando la organización de la información.
- **Control eficiente de la capacidad de transporte**
Permite monitorear la ocupación de cada bus de forma actualizada, evitando la sobrecarga de pasajeros y garantizando el cumplimiento de la capacidad máxima establecida para cada unidad.
- **Optimización de rutas y horarios**

Facilita la planificación y administración de rutas y horarios de transporte, permitiendo una mejor distribución de los buses entre las sedes universitarias y reduciendo tiempos de espera para los estudiantes.

- **Gestión integral de recursos**
Centraliza la administración de rutas, buses, conductores y pasajeros en un solo sistema, permitiendo una asignación más eficiente de los recursos disponibles.
- **Generación de reportes y estadísticas**
Proporciona reportes automáticos y estadísticas sobre el uso del servicio, como la ruta más utilizada, el horario de mayor demanda, la capacidad utilizada y el ranking de rutas, facilitando la supervisión administrativa y la toma de decisiones.
- **Análisis de la demanda del servicio**
Permite identificar patrones de uso del transporte universitario, facilitando la redistribución de unidades y la planificación de nuevas rutas o modificaciones en los horarios según la demanda.
- **Persistencia de la información**
Almacena los datos en archivos de texto, permitiendo conservar la información registrada, reducir pérdidas de datos y agilizar la generación de reportes administrativos.
- **Mejor experiencia para los estudiantes**
Brinda un servicio de transporte más organizado, seguro y eficiente, permitiendo a los estudiantes acceder a información actualizada sobre rutas, horarios y disponibilidad de espacios.
- **Apoyo a la toma de decisiones**
La información estadística generada por el sistema proporciona indicadores que ayudan a las autoridades universitarias a mejorar continuamente el servicio de transporte y optimizar la asignación de recursos.

2.8. Alcance del Sistema

El Sistema Inteligente de Control de Transporte Universitario tendrá como alcance la administración y control de los principales procesos relacionados con el servicio de transporte entre las sedes de la universidad. El sistema permitirá registrar, consultar, actualizar y gestionar la información necesaria para optimizar la operación del servicio.

- **Gestión de rutas y paraderos**
Permitirá registrar, modificar y consultar las rutas universitarias, incluyendo los paraderos que conforman cada recorrido, la capacidad máxima y el estado de disponibilidad de cada ruta.
- **Gestión de horarios**
Administrar los horarios de salida de los buses mediante una matriz bidimensional dinámica, permitiendo visualizar la ocupación de cada ruta en los distintos horarios establecidos.
- **Registro y control de pasajeros**
Permitirá registrar estudiantes en las rutas disponibles, verificando que la capacidad máxima de cada unidad no sea excedida y manteniendo actualizado el número de pasajeros registrados.
- **Gestión de unidades de transporte**
Administrará la información de los buses, permitiendo registrar las unidades disponibles y asignarlas a las diferentes rutas del servicio.
- **Gestión de conductores**
Permitirá registrar a los conductores y asociarlos con las unidades de transporte correspondientes para una adecuada organización del servicio.
- **Control de capacidad y ocupación**
Supervisar la ocupación de cada unidad de transporte, evitando la sobrecarga de pasajeros y mostrando la capacidad disponible en cada ruta.
- **Estadísticas del sistema**
Generará información estadística relevante, como la ruta más utilizada, la ruta menos utilizada, el horario de mayor demanda, el promedio de pasajeros, el porcentaje de ocupación y el ranking de rutas.

- Reportes administrativos

Permitirá generar reportes sobre rutas activas, pasajeros registrados, capacidad restante, historial de pasajeros y estadísticas generales del servicio de transporte.

- Gestión de archivos

Almacenará y recuperará la información del sistema mediante archivos de texto, garantizando la persistencia de los datos registrados y facilitando futuras consultas.

2.9. Requerimientos Funcionales

ID	Actor	Descripción	Prioridad
RF01	Administrador	El sistema debe permitir registrar una nueva ruta con código, nombre, paraderos y capacidad máxima.	Alta
RF02	Administrador	El sistema debe validar que no se registre una ruta con código duplicado.	Alta
RF03	Administrador	El sistema debe validar que la capacidad máxima ingresada sea un valor numérico válido mayor a cero.	Alta
RF04	Administrador	El sistema debe permitir modificar los paraderos, la capacidad o el estado de una ruta existente.	Alta
RF05	Administrador	El sistema debe permitir modificar los paraderos, la capacidad o el estado de una ruta existente.	Media
RF06	Administrador	El sistema debe permitir crear una matriz dinámica de horarios indicando el número de rutas y el número de horarios.	Alta
RF07	Administrador	El sistema debe permitir registrar la ocupación de pasajeros por ruta y horario dentro de la matriz.	Alta
RF08	Administrador	El sistema debe permitir visualizar la matriz de horarios con la ocupación de cada ruta por franja horaria.	Media
RF09	Administrador	El sistema debe permitir registrar unidades (buses) con placa.	Media
RF10	Administrador	El sistema debe permitir asignar una unidad disponible a una ruta específica.	Media
RF11	Administrador	El sistema debe permitir visualizar el listado de unidades y su disponibilidad.	Baja
RF12	Administrador	El sistema debe permitir registrar conductores con DNI y nombre.	Baja

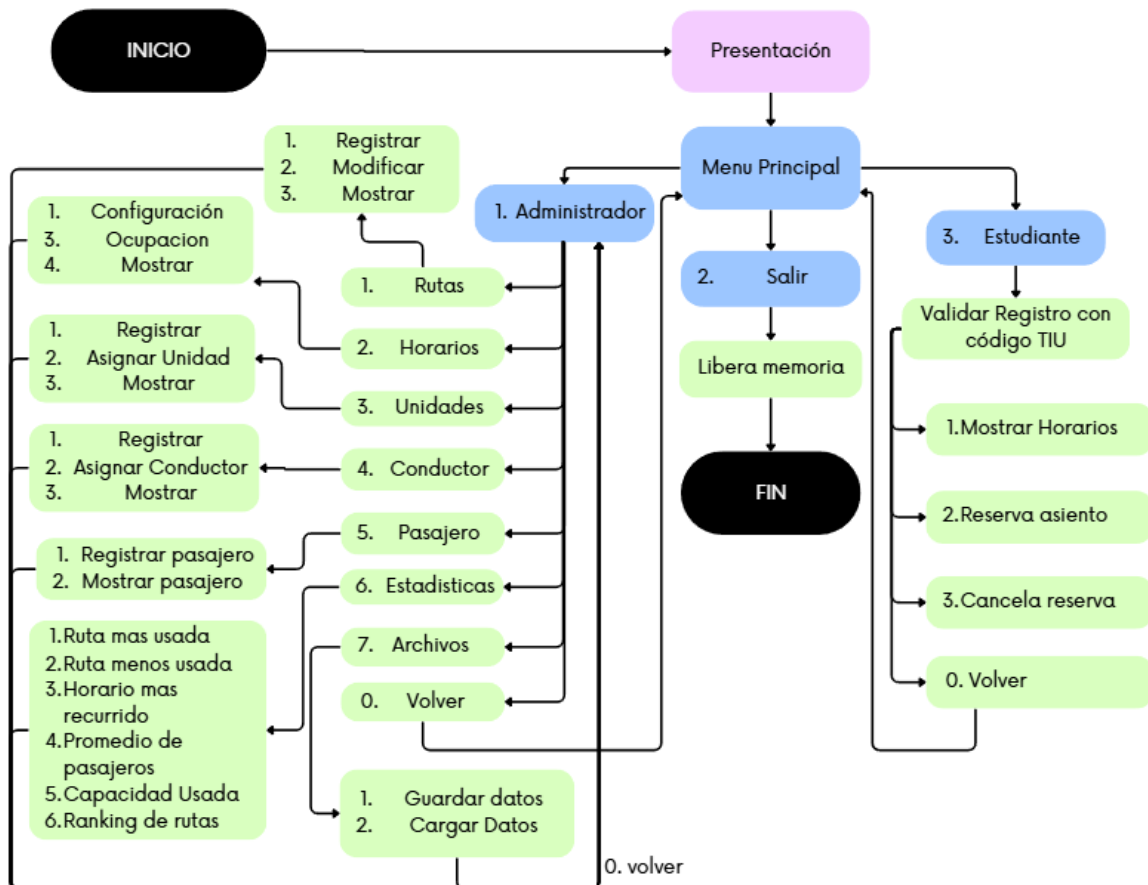
ID	Actor	Descripción	Prioridad
RF13	Administrador	El sistema debe permitir asignar un conductor a una unidad de transporte.	Baja
RF14	Administrador	El sistema debe permitir visualizar el listado de conductores y su unidad asignada.	Baja
RF15	Administrador	El sistema debe calcular y mostrar la ruta con mayor cantidad de pasajeros registrados.	Media
RF16	Administrador	El sistema debe calcular y mostrar la ruta con menor cantidad de pasajeros registrados.	Media
RF17	Administrador	El sistema debe calcular y mostrar el horario con mayor demanda de pasajeros.	Media
RF18	Administrador	El sistema debe calcular el promedio de pasajeros por ruta y el promedio diario general.	Media
RF19	Administrador	El sistema debe calcular y mostrar el porcentaje de capacidad utilizada por cada ruta.	Media
RF20	Administrador	El sistema debe generar un ranking de rutas ordenado según la cantidad de pasajeros registrados.	Media
RF21	Administrador	El sistema debe permitir guardar la información de rutas, pasajeros, horarios, unidades y conductores en archivos de texto.	Alta
RF22	Administrador	El sistema debe permitir cargar la información previamente guardada desde los archivos de texto.	Alta
RF24	Administrador	El sistema debe permitir registrar un pasajero ingresando código universitario, nombre y facultad.	Alta
RF25	Administrador	El sistema debe mostrar las rutas disponibles con su ocupación actual al momento de registrar un pasajero.	Alta
RF26	Administrador	El sistema debe validar que la ruta seleccionada no exceda su capacidad máxima antes de completar el registro.	Alta
RF27	Administrador	El sistema debe incrementar automáticamente el contador de pasajeros de la ruta y del horario al registrar un pasajero.	Alta
RF28	Administrador	El sistema debe permitir visualizar el historial completo de pasajeros registrados con su código, nombre y facultad.	Media
RF29	Estudiante	El sistema debe permitir registrar a un pasajero ingresando código universitario, nombre, facultad y ruta seleccionada.	Alta
RF30	Estudiante	El sistema debe validar que la ruta seleccionada no	Alta

ID	Actor	Descripción	Prioridad
		exceda su capacidad máxima antes de completar el registro.	
RF31	Estudiante	El sistema debe incrementar automáticamente el contador de pasajeros de la ruta y del horario al completar un registro exitoso.	Alta
RF32	Estudiante	El sistema debe permitir consultar las rutas universitarias disponibles.	Media
RF33	Estudiante	El sistema debe permitir consultar la matriz de horarios y su ocupación.	Media
RF34	Estudiante	El sistema debe permitir consultar las unidades (buses) asignadas a las rutas.	Baja
RF35	Estudiante	El sistema debe permitir consultar la capacidad restante disponible por ruta.	Media
RF36	Ambos	El sistema debe presentar un menú principal que permite acceder de forma diferenciada al perfil de Administrador o de Estudiante.	Alta
RF37	Ambos	El sistema debe validar que las opciones ingresadas en los menús sean valores numéricos, evitando fallos por entradas inválidas.	Alta

2.10. Ejemplo Reales

- UnCo Carpool: Conecta estudiantes de la Universidad de Lima para compartir viajes seguros y económicos, validando usuarios con correo institucional.(Universidad de lima, 2023)
- UniRide: Es una plataforma exclusiva para estudiantes universitarios en Arequipa, que conecta conductores con compañeros para viajes seguros, cómodos y accesibles.(UniRide,s.f)
- ByTheWay: Es una plataforma de carpooling exclusiva para universitarios, que reduce costos, emisiones y tiempos de viaje.(ByTheWay, 2025)

2.11. Algoritmo General del Proceso Principal



3. Capítulo 2: Estructuras, Archivos y Funciones

3.1. Estructura de Datos

El sistema utiliza cinco estructuras (struct). Cuatro de ellas representan las entidades del negocio (Ruta, Pasajero, Conductor y Unidad), y una quinta estructura, Sistema, agrupa a las demás junto con sus contadores y arreglos, funcionando como el contenedor principal de datos que se pasa por referencia entre las funciones del programa.

- Estructura Ruta

Campo	Tipo de dato	Descripción
codigo	char[10]	Código que identifica la ruta (arreglo de caracteres).
nombre	string	Nombre de la ruta.
paraderos	string	Paraderos asociados a la ruta.
capacidadMax	int	Capacidad máxima de pasajeros de la ruta.

Campo	Tipo de dato	Descripción
pasajerosActls	int	Cantidad actual de pasajeros con asiento reservado en la ruta.
estado	bool	Indica si la ruta está activa o inactiva.

- Estructura Pasajero

Campo	Tipo de dato	Descripción
codigo	string	Código universitario del pasajero.
nombre	string	Nombre del pasajero.
facultad	string	Facultad a la que pertenece el pasajero.
codigoRuta	string	Código de la ruta reservada por el pasajero. Queda vacío ("") si no tiene reserva activa.

- Estructura Conductor

Campo	Tipo de dato	Descripción
codigo	char[10]	Código que identifica al conductor.
nombre	string	Nombre del conductor.
placaUnidad	char[10]	Placa de la unidad asignada al conductor. Queda vacío si no tiene unidad asignada.

- Estructura Unidad

Campo	Tipo de dato	Descripción
placa	char[10]	Placa de la unidad (bus).
capacidad	int	Capacidad máxima de pasajeros de la unidad.
codigoRuta	char[10]	Código de la ruta a la que fue asignada la unidad. Queda vacío si no tiene ruta asignada.

- Estructura Sistema

Esta estructura centraliza todos los datos del programa. Se declara una única variable Sistema sist en main() y se pasa por referencia (Sistema&) a la mayoría de las funciones, evitando el uso de variables globales.

Campo	Tipo de dato	Descripción
rutas	Ruta*	Arreglo dinámico de rutas registradas.
numRutas	int	Cantidad actual de rutas registradas.
pasajeros	Pasajero*	Arreglo dinámico de pasajeros registrados.
numPasajeros	int	Cantidad actual de pasajeros registrados.
conductores	Conductor*	Arreglo dinámico de conductores registrados.
numConductores	int	Cantidad actual de conductores registrados.
unidades	Unidad*	Arreglo dinámico de unidades registradas.
numUnidades	int	Cantidad actual de unidades registradas.
matrizHorarios	int**	Matriz dinámica de ocupación de pasajeros por ruta y horario.
nombresHorarios	string*	Arreglo dinámico con el nombre/hora de cada horario configurado.
numHorarios	int	Cantidad de horarios configurados actualmente.

3.2. Archivos de Persistencia de Datos

Archivo	Contenido	Formato de ejemplo
rutas.txt	codigo, nombre, paraderos, capacidadMax, pasajerosActls, estado	RT01
pasajeros.txt	codigo, nombre, facultad, codigoRuta, posHorario	U20251O828
horarios.txt	Número de horarios, nombres y matriz de ocupación fila por fila	4 / 7:00am / 9:00am / 20

3.3. Funciones Principales del Sistema

El programa está dividido en 34 funciones (sin contar main()), organizadas en submenús por entidad. A continuación se detalla las responsabilidades y el impacto técnico de las funciones modulares implementadas en el código:

Nº	Función	Descripción
1	Presentacion()	Muestra la pantalla de bienvenida con el nombre del proyecto y los integrantes.
2	registrarRuta(Sistema&)	Registra una nueva ruta; inicializa pasajerosActls en 0 y estado en true.
3	mostrarRuta(Sistema&)	Muestra el listado completo de rutas con todos sus campos.
4	modificarRuta(Sistema&)	Busca una ruta por código y permite modificar paraderos, capacidad o estado.
5	menuRutas(Sistema&)	Submenú que agrupa registrar, mostrar y modificar rutas.
6	configHorar(Sistema&)	Configura la cantidad de horarios y sus nombres; inicializa la matriz en 0.
7	matrizHorar(Sistema&)	Muestra la matriz de horarios con la ocupación actual por ruta y horario.
8	menuHorarios(Sistema&)	Submenú que agrupa configurar horarios y ver la matriz.
9	registUni(Sistema&)	Registra una unidad con placa y capacidad, validando capacidad mayor a 0.
10	asignarUniRut(Sistema&)	Busca una unidad por placa y una ruta por código, y asigna la ruta a la unidad.
11	mostrarUnid(Sistema&)	Muestra el listado de unidades con su ruta asignada o "Sin asignar".

Nº	Función	Descripción
12	menuUnidades(Sistema&)	Submenú que agrupa registrar, asignar y ver unidades.
13	registConduct(Sistema&)	Registra un conductor con código y nombre; inicializa placaUnidad vacío.
14	asignarConductUnid(Sistema&)	Busca un conductor por código y una unidad por placa, y los vincula.
15	mostrarConduct(Sistema&)	Muestra el listado de conductores con su unidad asignada.
16	menuConductores(Sistema&)	Submenú que agrupa registrar, asignar y ver conductores.
17	registrPasaj(Sistema&)	Registra un pasajero con código, nombre y facultad; inicializa codigoRuta vacío y posHorario en -1.
18	mostrPasaj(Sistema&)	Muestra el historial de pasajeros registrados y el total.
19	menuPasajeros(Sistema&)	Submenú que agrupa registrar y mostrar pasajeros.
20	rutaMasUsada(Sistema&)	Muestra la ruta con mayor pasajerosActls usando strcpy_s y c_str().
21	rutaMenosUsada(Sistema&)	Muestra la ruta con menor pasajerosActls.
22	horarMasConcur(Sistema&)	Suma cada columna de la matriz y muestra el horario con mayor total.
23	promedioPasaj(Sistema&)	Calcula el promedio de pasajeros por horario de cada ruta y el promedio general.
24	capacUtili(Sistema&)	Calcula asientos disponibles y porcentaje de ocupación por ruta.
25	rankingRutas(Sistema&)	Ordena rutas por pasajerosActls con burbuja sobre arreglo auxiliar dinámico; libera con delete[].

Nº	Función	Descripción
26	menuEstadisticas(Sistema&)	Submenú que agrupa las seis funciones de estadísticas.
27	guardarDatos(Sistema&)	Escribe rutas, pasajeros y horarios en archivos .txt con ofstream y separador
28	cargarDatos(Sistema&)	Lee rutas, pasajeros y horarios desde archivos .txt con ifstream, separando por
29	menuArchivos(Sistema&)	Submenú que agrupa guardar y cargar datos.
30	menuAdministrador(Sistema&)	Menú principal del Administrador; enlaza con todos los submenús.
31	reservarAsiento(Sistema&, int)	Valida capacidad de ruta y cupo del bus (maxpasaj); actualiza pasajerosActls, posHorario y matrizHorarios.
32	cancelarReserva(Sistema&, int)	Cancela la reserva activa; decrementa pasajerosActls y la celda de la matriz; limpia codigoRuta y posHorario.
33	menuEstudiante(Sistema&)	Valida código universitario y muestra submenú: ver horarios, reservar y cancelar reserva.
34	main()	Reserva memoria dinámica, inicializa el sistema, muestra el menú principal y libera memoria al finalizar.

3.4. Relación entre requerimientos y funciones

Funcionalidad	Función(es) relacionada(s)	Descripción
Registrar, listar y modificar rutas	registrarRuta(), mostrarRuta(), modificarRuta(), menuRutas()	Alta, consulta y edición de rutas.
Configurar y consultar horarios	configHorar(), matrizHorar(), menuHorarios()	Crea la matriz dinámica y muestra la ocupación.
Registrar, asignar y listar unidades	registUni(), asignarUniRut(), mostrarUnid(), menuUnidades()	Gestiona unidades y su relación con rutas.
Registrar, asignar y listar conductores	registConduct(), asignarConductUnid(),	Gestiona conductores y su relación con las unidades.

	mostrarConduct(), menuConductores()	
Registrar y listar pasajeros	registrPasaj(), mostrPasaj(), menuPasajeros()	Alta administrativa de pasajeros e historial.
Estadísticas	rutaMasUsada(), rutaMenosUsada(), horarMasConcur(), promedioPasaj(), capacUtili(), rankingRutas(), menuEstadisticas()	Calcula y muestra todos los indicadores del sistema
Guardar y cargar datos	guardarDatos(), cargarDatos(), menuArchivos()	Persistencia de datos en archivos .txt.
Menú por perfil	main(), menuAdministrador(), menuEstudiante()	Separa el flujo de Administrador y Estudiante.
Reservar y cancelar asiento	reservarAsiento(), cancelarReserva()	Permite al estudiante autogestionar su reserva.

3.5. Librerías utilizadas

Librería	Uso en el código
#include <iostream>	Entrada y salida estándar (cin, cout) en todos los menús y funciones.
#include <string>	Tipo string usado en nombre, paraderos, facultad, codigoRuta, nombresHorarios.
#include <cstring>	Funciones strcmp() y strcpy_s() para comparar y copiar los campos char[].
#include <fstream>	Lectura y escritura de archivos de texto (ofstream para guardar, ifstream para cargar).
using namespace std;	Evita anteponer std:: a cout, cin, string, etc.

4. Capítulo 3: Listado de funcionalidades cumplidas y Manual de Usuario

4.1. Listado de funcionalidades cumplidas

Nº	Funcionalidad requerida	Estado	Función(es) que la implementan
1	Registra rutas universitarias	Cumplido	registrarRuta()
2	Mostrar rutas activas e inactivas	Cumplido	mostrarRuta()

Nº	Funcionalidad requerida	Estado	Función(es) que la implementan
3	Modificar paraderos, capacidad y estado de ruta	Cumplido	modificarRuta()
4	Administrar horarios con matriz bidimensional dinámica	Cumplido	configHorar(), matrizHorar()
5	Mostrar matriz de horarios con ocupación	Cumplido	matrizHorar()
6	Registrar pasajeros con código, nombre y facultad	Cumplido	registrPasaj()
7	Controlar capacidad máxima de la ruta al reservar	Cumplido	reservarAsiento()
8	Controlar capacidad máxima del bus por horario (maxpasaj=50)	Cumplido	reservarAsiento()
9	Incrementar contador de pasajeros de ruta y horario al reservar	Cumplido	reservarAsiento()
10	Cancelar reserva de asiento	Cumplido	cancelarReserva()
11	Gestionar unidades (registrar, asignar, mostrar)	Cumplido	registUni(), asignarUniRut(), mostrarUnid()
12	Gestionar conductores (registrar, asignar, mostrar)	Cumplido	registConduct(), asignarConductUnid(), mostrarConduct()
13	Mostrar ruta más utilizada	Cumplido	rutaMasUsada()
14	Mostrar ruta menos utilizada	Cumplido	rutaMenosUsada()
15	Mostrar horario más concurrido	Cumplido	horarMasConcur()
16	Calcular promedio de pasajeros por ruta y general	Cumplido	promedioPasaj()
17	Mostrar capacidad utilizada y disponible por ruta	Cumplido	capacUtili()
18	Ranking de rutas ordenado por pasajeros (método burbuja)	Cumplido	rankingRutas()
19	Guardar datos en archivos de texto	Cumplido	guardarDatos()
20	Cargar datos desde archivos	Cumplido	cargarDatos()

Menú Principal

```
MENU PRINCIPAL
1. Administrador
2. Estudiante
3. Salir
Ingrese una opcion: _
```

Menú Administrador

```
MENU ADMINISTRADOR
1. Rutas
2. Horarios
3. Unidades
4. Conductores
5. Pasajeros.
6. Estadísticas
7. Archivos
0. Volver
Seleccione: _
```

Menú Rutas

```
RUTAS
1. Registrar ruta
2. Mostrar rutas
3. Modificar ruta
0. Volver
Seleccione: _
```

Menú Horarios

```
HORARIOS
1. Configurar horarios
2. Ver matriz de horarios
0. Volver
Seleccione: _
```

Menú Unidades

```
UNIDADES
1. Registrar unidad
2. Asignar unidad a ruta
3. Ver unidades
0. Volver
Seleccione: _
```

Menú Conductores

```
CONDUCTORES
1. Registrar conductor
2. Asignar conductor a unidad
3. Ver conductores
0. Volver
Seleccione: _
```

Menú Pasajeros

```
PASAJEROS
1. Registrar pasajero
2. Mostrar pasajeros
0. Volver
Seleccione: _
```

Menú Estadísticas

```
ESTADISTICAS
1. Ruta mas utilizada
2. Ruta menos utilizada
3. Horario mas concurrido
4. Promedio de pasajeros
5. Capacidad utilizada
6. Ranking de rutas
0. Volver
Seleccione: _
```

Menú Archivo

Menú Estudiante


```

    ARCHIVOS
1. Guardar datos
2. Cargar datos
0. Volver
Seleccione: █

```

```

Ingrese su codigo universitario: U202510828

Bienvenido, Miguel Escobar!

    MENU ESTUDIANTE
1. Ver horarios
2. Reservar asiento
3. Cancelar reserva
0. Volver
Seleccione: █

```

4.2.3. Menú Administrador - Funciones

- Rutas
 - Registrar Ruta

```

REGISTRAR RUTA
Codigo de ruta (formato RUTXX): RUD00
Formato invalido. Ejemplo: 'RUT01'.
Codigo de ruta (formato RUTXX): RUT01
Nombre de ruta : Monterrico-Villa
Paraderos      : Monterrico- Av.Encalada-Av.Benavides-Vill
Capacidad max  : 200
Ruta registrada exitosamente.

```

Para registrar una nueva ruta, el sistema solicitará los siguientes datos:

Código de ruta: Formato RUTXX, donde XX es un número de dos dígitos (ejemplo: RUT01). No se aceptan códigos vacíos, con formato incorrecto ni duplicados.

Nombre de ruta: Se recomienda indicar la sede de origen y destino separadas por un guión (ejemplo: San Miguel - Monterrico). No se aceptan campos vacíos.

Paraderos: Nombres de los paraderos separados por guiones (ejemplo: San Miguel - Miraflores - Monterrico). No se aceptan campos vacíos.

Capacidad máxima: Número entero mayor a cero que representa el total de pasajeros que pueden reservar asiento en esa ruta. No se aceptan valores de cero o negativos.

Si todos los datos son válidos, el sistema mostrará *"Ruta registrada exitosamente"* y la ruta quedará activa con cero pasajeros.

- Mostrar Rutas

```
Codigo: RUT03
Nombre: SanIsidro-Villa
Paraderos: SanI-Av.Salaverry-Av.JavierPrado-Av.Ejercito-Villa
Capacidad : 200
Pasajeros : 0
Estado    : Activa

Codigo: RUT04
Nombre: Villa-SanIsidro
Paraderos: Villa-Av.Ejercito-Av.JavierPrado-Av.Salaverry-SanI
Capacidad : 200
Pasajeros : 0
Estado    : Activa

Codigo: RUT05
Nombre: SanMiguel-Villa
Paraderos: Sanmiguel-Villa
Capacidad : 100
Pasajeros : 0
Estado    : Inactiva
```

Muestra los datos de las rutas.

- Modificar Ruta

```
MODIFICAR RUTA

Ingrese el codigo de ruta a modificar: RUT05
Ruta encontrada: SanMiguel-Villa
1. Modificar Paraderos.
2. Estado de Ruta (Act/Inact).
0. VolverSeleccione: 2
Estado de ruta: Inactiva.
```

Para modificar una ruta, el sistema solicitará el código de la ruta a modificar. Si existe, se mostrará un submenú con tres opciones: actualizar los paraderos, modificar la capacidad máxima (debe ser mayor a cero) y alternar el estado entre Activa e Inactiva. Si el código no existe, el sistema mostrará el mensaje *"Ruta no encontrada"*.

- Horarios
 - Configurar Horario

```

CONFIGURAR HORARIOS
Cuantos horarios desea registrar (max: 6)
Ingrese este formato ejemplo: 6:00am: 4
Horario 18:00am
Horario 211:00am
Horario 313:00am
Horario 417:00pm
Horarios configurados correctamente.

```

Para configurar los horarios, el sistema solicitará la cantidad de horarios a registrar, con un máximo de 6. Luego pedirá ingresar el nombre de cada franja horaria en formato HH:MMam/pm (ejemplo: 08:00am, 02:00pm). Una vez configurados, la matriz de horarios se inicializa en cero para todas las rutas registradas.

- Ver Matriz Horario

```

MATRIZ DE HORARIOS
      8:00am 11:00am 13:00am 17:00pm
Monterrico-Villa      1      0      2      1
Villa-Monterrico      0      0      2      1
SanIsidro-Villa       0      1      0      1
Villa-SanIsidro        0      0      1      0
SanMiguel-Villa        0      1      0      0

```

Se muestra la matriz de horarios en consola, donde las columnas representan las rutas intersecciones y los bloques de horas disponibles, mientras que las filas registran la cantidad de alumnos inscritos en cada servicio.

- Unidades
 - Registrar Unidad

```

REGISTRAR UNIDADES
Placa (formato HHH-XXX): AEC-443
Unidad registrada correctamente.

```

Al registrar una unidad, se exige un formato obligatorio de placa: 3 letras mayúsculas, un guión y 3 números enteros positivos (ej. ABC-123). Sin cumplir con este formato exacto, el sistema bloqueará el registro del bus.

- Asignar Unidad a Ruta

```
ASIGNAR UNIDAD A RUTA  
Placa de la Unidad: ABC-123  
Rutas disponibles.  
1. RUT01 - Monterrico-Villa  
2. RUT02 - Villa-Monterrico  
3. RUT03 - SanIsidro-Villa  
4. RUT04 - Villa-SanIsidro  
5. RUT05 - SanMiguel-Villa  
Codigo de ruta a asignar: RUT01  
Unidad ABC-123 asignada a Monterrico-Villa.
```

Para asignar un bus, se ingresa la placa de la unidad y luego se selecciona el código de la ruta desde una lista en pantalla. Al finalizar, el sistema muestra el mensaje de confirmación 'Unidad ABC-123 asignada a Monterrico-Villa'.

- Ver Unidades

```
LISTADO DE UNIDADES  
  
Placa      : ABC-123  
Capacidad  : 50  
Ruta asign.: RUT01  
-----  
  
Placa      : DEF-456  
Capacidad  : 50  
Ruta asign.: RUT02  
-----  
  
Placa      : GHI-789  
Capacidad  : 50  
Ruta asign.: RUT03  
-----  
  
Placa      : AEC-443  
Capacidad  : 50  
Ruta asign.: Sin asignar  
-----
```

Muestra la lista de todas las unidades registradas, detallando su placa, su capacidad fija (50 alumnos para todos los buses) y si cuentan o no con una ruta asignada actualmente.

- Conductores
 - Registrar Conductor

```
REGISTRAR CONDUCTOR
Codigo (formato CDTXX): CDT02
Nombre : Frank Castle
Conductor registrado exitosamente.
```

Al registrar un conductor, se exige un formato obligatorio de código: CDT y 2 números enteros positivos (ej. CDT-13). Sin cumplir con este formato exacto, el sistema bloqueará el registro del bus. Finalmente pedirá el nombre del conductor.

- Asignar Conductor a Unidad

```
Codigo del conductor: CDT01
Unidades disponibles:
1. ABC-123 (capacidad max.: 50)
2. DEF-456 (capacidad max.: 50)
3. GHI-789 (capacidad max.: 50)
4. AEC-443 (capacidad max.: 50)
5. FDF-434 (capacidad max.: 50)
Placa de la unidad: ABC-123
Conductor Alfredo Rodriguez asignado a la unidadABC-123.
```

Para asignar un conductor a un bus, se ingresa el código del conductor y luego se selecciona el código de la placa desde una lista en pantalla. Al finalizar, el sistema muestra el mensaje de confirmación 'Conductor Alfredo Rodriguez asignado a la unidad ABC-123'.

- Ver Conductores

```
LISTADO DE CONDUCTORES

Codigo : CDT01
Nombre : Alfredo Rodriguez
Unidad (placa): ABC-123

Codigo : CDT02
Nombre : Frank Castle
Unidad (placa): Conductor no asignado.
```

Muestra la lista de los conductores registrados, detallando su código, el nombre del conductor y si cuentan o no con una unidad asignada actualmente.

- Pasajeros
 - Registrar Pasajero

```
REGISTRAR PASAJERO
Codigo universitario (ej: U20241J117): U202510828
Nombre : Miguel Escobar
Facultad : Ingenieria

Pasajero registrado exitosamente.
```

Al registrar un pasajero, se exige un código obligatorio de 10 dígitos: inicia con la 'U', seguido de números hasta la posición 7 (que acepta número o letra) y finaliza con 3 números (ej. U202510828). Si no cumple este formato exacto, el sistema bloquea el registro. Finalmente, se ingresan el nombre y la facultad del estudiante.

- Mostrar Pasajero

```
HISTORIAL DE PASAJEROS

Codigo: U202510828
Nombre: Miguel Escobar
Facultad: Ingenieria

Codigo: U20241J117
Nombre: Mao Ramirez
Facultad: Ingenieria

Codigo: U20251L120
Nombre: Dayara Zelada
Facultad: Ingenieria
```

Se muestra el código, nombre y facultad de todos los estudiantes registrados en el sistema.

- Estadísticas
 - Ruta más utilizada

```
RUTA MAS USADA
Ruta : Monterrico-Villa
Pasajeros: 4
```

Muestra la ruta con mayor cantidad de pasajeros con asiento reservado actualmente, indicando el nombre de la ruta y el total de pasajeros.

- Ruta menos utilizada

```
RUTA MENOS USADA
Ruta: Villa-SanIsidro
Pasajeros: 1
```

Muestra la ruta con menor cantidad de pasajeros con asiento reservado, útil para identificar rutas con baja demanda.

- Horario más concurrido

```
HORARIO MAS CONCURRIDO
Horario : 13:00am
Total de pasajeros: 5
```

Recorre la matriz de horarios sumando los pasajeros de todas las rutas por cada franja horaria, y muestra el horario con mayor total acumulado.

- Promedio de pasajeros

```
PROMEDIO DE PASAJEROS

Ruta : Monterrico-Villa
Promedio por horario: 1 pasajeros

Ruta : Villa-Monterrico
Promedio por horario: 0.75 pasajeros

Ruta : SanIsidro-Villa
Promedio por horario: 0.5 pasajeros

Ruta : Villa-SanIsidro
Promedio por horario: 0.25 pasajeros

Ruta : SanMiguel-Villa
Promedio por horario: 0.25 pasajeros

Promedio general de pasajeros: 0.55
```

Calcula por cada ruta el promedio de pasajeros dividiendo la suma de ocupación de todos sus horarios entre la cantidad de horarios configurados. Adicionalmente muestra el promedio general del sistema, dividiendo el total de pasajeros de toda la matriz entre el número total de celdas (rutas × horarios).

- Capacidad utilizada

```
Ruta : Monterrico-Villa
Capacidad : 200
Ocupados : 4
Disponibles: 196
Ocupacion : 2%

Ruta : Villa-Monterrico
Capacidad : 200
Ocupados : 3
Disponibles: 197
Ocupacion : 1.5%
```

Muestra por cada ruta la cantidad de asientos ocupados, los asientos disponibles y el porcentaje de ocupación respecto a su capacidad máxima.

- Ranking de rutas

```
RANKING DE RUTAS

N.1
Ruta: Monterrico-Villa
Pasajeros: 4

N.2
Ruta: Villa-Monterrico
Pasajeros: 3

N.3
Ruta: SanIsidro-Villa
Pasajeros: 2

N.4
Ruta: Villa-SanIsidro
Pasajeros: 1

N.5
Ruta: SanMiguel-Villa
Pasajeros: 1
```

Ordena todas las rutas de mayor a menor según la cantidad de pasajeros registrados, utilizando el algoritmo de burbuja, y muestra el listado numerado.

- Archivo
 - Guardar Datos

```
El archivo 'rutas.txt' se ha aperturado correctamente.
'rutas.txt' guardado correctamente.
El archivo 'pasajeros.txt' se ha aperturado correctamente.
'pasajeros.txt' guardado.
El archivo 'horarios.txt' se ha aperturado correctamente.
```

Almacena el estado actual del sistema en tres archivos de texto: rutas.txt, pasajeros.txt y horarios.txt. Cada archivo guarda los registros separando los campos con el carácter |. Si los archivos ya existen, serán sobrescritos con la información más reciente, por lo que se recomienda guardar antes de cerrar el programa para no perder los datos.

- Cargar Datos

```
Rutas cargadas: 5
Pasajeros cargados: 13
Horarios cargados: 4

Datos cargados correctamente.
```

Lee los archivos rutas.txt, pasajeros.txt y horarios.txt previamente guardados y restaura toda la información al sistema, incluyendo rutas, pasajeros y la matriz de ocupación de horarios. Se recomienda usar esta opción al inicio de cada sesión para continuar trabajando con los datos registrados anteriormente.

4.2.4. Menú Estudiante - Funciones

- Ver horarios

MATRIZ DE HORARIOS				
	8:00am	11:00am	13:00am	17:00pm
Monterrigo-Villa	1	0	2	1
Villa-Monterrigo	0	0	2	1
SanIsidro-Villa	0	1	0	1
Villa-SanIsidro	0	0	1	0
SanMiguel-Villa	0	1	0	0

Se muestra la matriz de horarios en consola, donde las columnas representan las rutas intersedes y los bloques de horas disponibles, mientras que las filas registran la cantidad de alumnos inscritos en cada servicio.

- Reservar asiento

```
----- RESERVAR ASIENTO -----  
Rutas disponibles:  
1. Monterrico-Villa  
2. Villa-Monterrico  
3. SanIsidro-Villa  
4. Villa-SanIsidro  
5. SanMiguel-Villa  
Seleccione numero de ruta: 1  
  
Horarios disponibles:  
1. 8:00am (0/50)  
2. 11:00am (0/50)  
3. 13:00am (0/50)  
4. 17:00pm (0/50)  
Seleccione horario: 1  
  
Asiento reservado en Monterrico-Villa - 8:00am
```

Al reservar asientos, se muestran las rutas disponibles con sus respectivos horarios, indicando entre paréntesis la cantidad de asientos ocupados sobre el total disponible (ej. 15/50).

- Cancelar reserva

```
----- CANCELAR RESERVA -----  
Ruta actual: SanMiguel-Villa  
Confirmar cancelacion (1=Si / 0=No): 1  
Reserva cancelada exitosamente.
```

Para cancelar la reserva, el sistema muestra la ruta actual en la que el alumno está inscrito y solicita una confirmación para proceder con la cancelación en tiempo real.

4.2.5. Validaciones y restricciones

El sistema incluye filtros en las entradas de datos para restringir ingresos incorrectos, garantizando el control, la validación y la visualización correcta de la información en cada módulo.

No está permitido el ingreso con espacio en blanco o un *enter*.

```
Nombre :  
El nombre no puede estar vacio.
```

Si en el menú de opciones se ingresa un número incorrecto o un carácter no válido, el programa bloqueará la entrada y repetirá la solicitud de forma persistente hasta que se digite una opción válida.

```
MENU PRINCIPAL  
1. Administrador  
2. Estudiante  
3. Salir  
Ingrese una opcion: 8  
Opcion invalida.
```

Si se intenta acceder a una opción de visualización sin haber ingresado datos previamente, el sistema mostrará un aviso de advertencia indicando que la lista o reporte se encuentra vacío

```
RUTAS  
1. Registrar ruta  
2. Mostrar rutas  
3. Modificar ruta  
0. Volver  
Seleccione: 2  
No hay rutas registradas.
```

```
MATRIZ DE HORARIOS  
No hay rutas registradas.
```

```
ASIGNAR UNIDAD A RUTA  
No hay espacio para mas unidades.
```

```
HORARIOS  
1. Configurar horarios  
2. Ver matriz de horarios  
0. Volver  
Seleccione: 1  
CONFIGURAR HORARIOS  
No hay rutas registradas, asegure de haber al menos una.
```

Cada código solicitado posee un formato obligatorio. Si el usuario ingresa un dato incorrecto, el sistema activa alertas específicas para impedir el registro y guiar el ingreso correcto:

```
REGISTRAR RUTA
Codigo de ruta (formato RUTXX): RUD00
Formato invalido. Ejemplo: 'RUT01'.
Codigo de ruta (formato RUTXX):
```

```
REGISTRAR UNIDADES
Placa (formato HHH-XXX): ABC-AA
Formato invalido. Ejemplo: 'ABC-123'.
Placa (formato HHH-XXX):
```

```
REGISTRAR CONDUCTOR
Codigo (formato CDTXX): CDTRR
Formato invalido. Ejemplo: 'CDT01'.
Codigo (formato CDTXX):
```

No se puede registrar un código ya existente.

```
REGISTRAR RUTA
Codigo de ruta (formato RUTXX): RUT02
Ya existe una ruta con ese codigo.
```

En registrar pasajeros, se debe cumplir el formato establecido del código de estudiante y no puede registrarse un código ya existente.

```
REGISTRAR PASAJERO
Codigo universitario (ej: U20241J117): U20241C003
Ya existe un pasajero con ese codigo.
Codigo universitario (ej: U20241J117):
```

```
REGISTRAR PASAJERO
Codigo universitario (ej: U20241J117): U20A23411
Formato invalido. Ejemplo: 'U20241J117'.
Codigo universitario (ej: U20241J117):
```

En el caso del menú estudiantes, el código de estudiante debe estar registrado en el sistema antes de poder reservar un asiento.

```
Ingrese su codigo universitario: U20241D004
Este usuario no esta registrado. Contactar al administrador.
```

4.2.6. Salir

```
Saliendo del sistema...
Nos vemos!

Memoria liberada correctamente.
```

5. Conclusiones

Escobar Cory, Miguel Andrée:

El desarrollo del Sistema Inteligente de Control de Transporte Universitario permitió aplicar de manera práctica los conceptos de estructuras, arreglos dinámicos y archivos de texto vistos a lo largo del curso. Trabajar con la estructura Ruta me ayudó a comprender cómo agrupar datos relacionados bajo un mismo nombre y cómo esto facilita el mantenimiento del código en comparación con el uso de variables sueltas.

Asimismo, dividir el sistema en funciones independientes, como registrar Ruta() y modificar Ruta(), reforzó la importancia de aplicar un diseño modular, ya que permitió aislar errores con mayor facilidad y realizar pruebas por partes en lugar de depender de una única función principal. Considero que este proyecto fortaleció mi capacidad para transformar un problema real, como la gestión del transporte universitario, en una solución estructurada aplicando los principios de programación aprendidos en el curso.

Ramírez Huamani, Mao Lenon:

Trabajar con una matriz dinámica bidimensional para representar la ocupación de pasajeros por ruta y horario me permitió comprender mejor el manejo de memoria dinámica en C++ y su utilidad para resolver problemas donde el tamaño de los datos no puede definirse de forma fija al iniciar el programa. Este proceso también evidenció la importancia de liberar correctamente la memoria reservada para evitar errores de ejecución.

Además, el análisis de las dependencias entre las rutas, los horarios y las unidades de transporte me ayudó a entender que un sistema de información no está compuesto por partes aisladas, sino por componentes interconectados cuyo correcto funcionamiento depende de validaciones consistentes en cada punto del flujo. Esta experiencia fortaleció mi criterio para diseñar soluciones que consideren tanto la lógica del programa como la coherencia de los datos que administra.

Rodriguez Quispe, Dario Francisco:

El programa demuestra que el uso de arreglos y matrices dinámicas en C++ es una solución eficiente para gestionar las rutas de la UPC, logrando un mejor control de los buses y facilitando los traslados de los estudiantes sin que el sistema se sature ya que el programa también cuenta con validaciones de entrada y límites.

En general, este trabajo me ha enseñado a aplicar estructuras y matrices dinámicas en C++ para resolver problemas reales de logística. Además, me ayudó a comprender la importancia de los reportes estadísticos en tiempo real, los cuales no solo mejoran el control del sistema actual, sino que sirven como una herramienta clave para tomar mejores decisiones operativas en el futuro.

Zelada Gutarate, Dayara Priyanka:

El uso de archivos de texto para guardar y recuperar la información de rutas, unidades, conductores y pasajeros permitió comprender cómo conservar datos persistentes sin depender de una base de datos, cumpliendo con los objetivos planteados en el curso de Lenguaje de Programación. Este proceso también hizo evidente la

importancia de verificar que un archivo exista o se pueda abrir correctamente antes de leerlo, para evitar fallos durante la ejecución del programa.

6. Recomendaciones

Escobar Cory, Miguel André:

Se recomienda, en una futura versión, migrar el almacenamiento de archivos de texto a una base de datos relacional, ya que esto permitiría manejar un mayor volumen de rutas y pasajeros sin depender de la lectura secuencial de archivos, además de facilitar consultas más complejas sobre la información registrada. Esto también permitiría implementar un control de concurrencia, útil si en el futuro varios administradores gestionan el sistema al mismo tiempo desde distintos equipos.

Ramírez Huamani, Mao Lenon:

Se recomienda incorporar una interfaz gráfica para el registro y consulta de rutas y horarios, en lugar de trabajar únicamente en consola, ya que esto mejoraría la experiencia del estudiante y facilita la visualización de la matriz de ocupación de manera más intuitiva. Una interfaz de este tipo también reduciría los errores de digitación al ingresar datos, al permitir el uso de listas desplegables y validaciones visuales en tiempo real.

Rodriguez Quispe, Dario Francisco:

Se recomienda ampliar las validaciones del sistema incorporando el control de horarios en conflicto entre rutas y la verificación de disponibilidad de unidades en tiempo real, con el fin de evitar asignaciones duplicadas y mejorar la confiabilidad del proceso administrativo. Aunque la lógica de archivos planos y el ordenamiento por burbuja que implementé funcionan muy bien para el tamaño actual del sistema, estas mejoras de control de errores harían que el programa sea mucho más fácil de usar por los estudiantes y evitarían cualquier caída si el usuario se equivoca al digitar.

Zelada Gutarate, Dayara Priyanka:

Se recomienda automatizar la generación de reportes en formatos adicionales, como hojas de cálculo, y añadir gráficos estadísticos de la demanda por ruta y horario, lo cual facilita el análisis de la información por parte de la administración del transporte universitario. También se sugiere programar el envío periódico de estos reportes, de manera que la información esté siempre disponible para la toma de decisiones sin depender de una solicitud manual.

7. Bibliografía y Citas

Facultad de Ciencias e Ingeniería PUCP. (2024, 12 de noviembre). El desafío de todo estudiante PUCP: la congestión vehicular de Lima. Pontificia Universidad Católica del Perú. <https://facultad-ciencias-ingenieria.pucp.edu.pe/2024/11/12/el-desafio-de-todo-estudiante-pucp-la-congestion-vehicular-de-lima/>

UniRide.(s.f.).Sobre Nosotros. UniRide. <https://uniride8.webnode.pe/sobre-nosotros/>

Universidad de Lima (2023, 26 de Abril). Un “carpool” para estudiantes Ulima. <https://www.ulima.edu.pe/centros-e-institutos/innova-ul/noticias/un-carpool-para-estudiantes-ulima>

BytheWay(2025).¿Que es BytheWays?.ByTheWays. <https://www.bythewaycarpool.com>

GreenLine. (2026, 9 Marzo). ¿Puede el caos del tráfico en Lima afectar el rendimiento académico de los universitarios?. <https://glperu.com/el-impacto-del-traffic-en-los-estudiantes/>

PIARC. (2023). Sistema de Transporte Inteligente. Manual de Gestión de Desastres.<https://disaster-management.piarc.org/es/respuesta-medidas-de-respuesta/sistema-de-transporte-inteligente>

8. Participación

Alumno	Código	% de Participación
Escobar Cory, Miguel Andrée	U20251O828	100%
Ramírez Huamani, Mao Lenon	U20241J117	100%
Rodriguez Quispe, Dario Francisco	U202523240	100%
Zelada Gutarate, Dayara Priyanka	U20251L120	100%